How-To Library

Informatica

Performance Tuning Guidelines for PowerExchange for Google BigQuery on the Spark Engine

© Copyright Informatica LLC 2019, 2021. Informatica and the Informatica logo are trademarks or registered trademarks of Informatica LLC in the United States and many jurisdictions throughout the world. A current list of Informatica trademarks is available on the web at https://www.informatica.com/trademarks.html

Abstract

This article describes the key hardware and Hadoop cluster parameters that you can tune to optimize the performance of PowerExchange for Google BigQuery for Spark. Learn how to optimize the performance of PowerExchange for Google BigQuery on Spark by tuning these parameters appropriately.

Supported Versions

- PowerExchange for Google BigQuery 10.4.0

Table of Contents

Overview. 2
General Best Practices. 2
Performance Tuning Parameters. 3
Tune the Hardware for the Hadoop Cluster based on Dataset. 3
Tune the Partitions per Core for a Read Operation. 4
Tune the Spark Engine. 4
Troubleshooting Spark Tuning Parameters. 6

Overview

Performance tuning is an iterative process in which you analyze the performance, use guidelines to estimate and define parameters that impact the performance, and monitor and adjust the results as required.

The article includes case studies that involve loading data into Google BigQuery from HDFS and reading data from Google BigQuery to HDFS using Dataproc 1.4.

Note: The performance testing results listed in this article are based on observations in an internal Informatica environment using data from real-world scenarios. The performance of PowerExchange for Google BigQuery might vary based on individual environments and other parameters even when you use the same data.

General Best Practices

To optimize the performance, consider the following best practices for Google BigQuery and Google Dataproc:

- Host Informatica domain on a Google Compute Engine virtual machine instance to optimize the Model Repository Service monitoring.
- Launch the Dataproc cluster in the region where Google BigQuery is configured.
- Make sure that you complete tuning the Spark engine. For more information on tuning the Spark engine, see "Tune the Spark Engine" on page 4.
- If the Google BigQuery file contains a high precision decimal data type, change the precision of decimal data type to low precision to reduce the size of decimal data.
- Informatica recommends to use Avro as the file format when you read data from or write data to Google BigQuery for better performance.
- Informatica recommends to use Google Dataproc as the Hadoop distribution when you read data from or write data to Google BigQuery for better performance.

Performance Tuning Parameters

You can optimize the performance of Google BigQuery mappings by tuning the following components:

- Tune the hardware for the Hadoop cluster based on dataset.
- Tune the Partitions per core for the read operation.
- Tune the Spark engine.

Tune the Hardware for the Hadoop Cluster based on Dataset

To optimize the performance, acquire the right type of hardware for your Data Engineering Integration environment. You can tune the hardware by configuring the Hadoop cluster based on different data sets when you run mappings on the Spark engine.

Small Data Set

Small data set is defined as a system with one master node and two worker nodes that loads 7.5 GB of data from a Google BigQuery data source to a HDFS target in a Dataproc cluster in 0:15:30 (hh:mm:ss) hours or from an HDFS source in a Dataproc cluster to a Google BigQuery target in 00:17:00 (hh:mm:ss) hours.

The following table shows the sizing outcome for default settings:

	Infrastructure	**Hadoop Services**
Services	Domain, MRS for Monitoring, DIS	Yarn Resource Manager, Node Manager
vCPUs	2	(4*1) + (4*2) = 12
Memory	8 GB	(4*1) + (12 *2) = 28 GB
Storage	14 GB	7.5 * 3 = 22.5 GB

Note: Total vCPUs = vCPUs of 1 Master + (vCPUs in core Node * No of Core Nodes).

Total Memory = (Master Node Process Memory + (Node Process Memory * Number of core Nodes)).

Total Storage = (I/P Data * Replication Factor)

Medium Data Set

Medium data set is defined as a system, with one master node and nine worker nodes, that loads 75 GB of data from a Google BigQuery data source to a HDFS target in a Dataproc cluster in 0:15:30 (hh:mm:ss) hours or from an HDFS source in a Dataproc cluster to a Google BigQuery target in 00:17:00 (hh:mm:ss) hours.

The following table shows the sizing recommendations for a medium data set:

	Infrastructure	**Hadoop Services**
Services	Domain, MRS for Monitoring, DIS	Yarn Resource Manager, Node Manager
vCPUs	2	(4*1) + (4*9) = 40
Memory	8 GB	(4*1) + (12*9) = 112 GB
Storage	14 GB	75*3 = 225 GB

Large Data Set

Large data set is defined as a system, with one master node and 45 worker nodes, that loads 375 GB of data from a Google BigQuery data source to a HDFS target in a Dataproc cluster in 0:15:30 (hh:mm:ss) hours or from an HDFS source in a Dataproc cluster to a Google BigQuery target in 00:17:00 (hh:mm:ss) hours.

The following table shows the sizing recommendation for a large data set:

	Infrastructure	Hadoop Services
Services	Domain, MRS for Monitoring, DIS	Yarn Resource Manager, Node Manager
vCPUs	2	(4*1) + (4*45) = 184
Memory	8 GB	(4*1) + (12*45) = 544 GB
Storage	14 GB	375*3 = 1125 GB

Note: The storage recommendation is the minimum storage required for HDFS replication, you need to add additional storage to accommodate other Informatica related operations.

Tune the Partitions per Core for a Read Operation

You can increase the performance when you read data from a large Google BigQuery table in optimized Spark mode.

When you run a Google BigQuery mapping in optimized Spark mode, you can configure the **Partitions per core** property to control the number of partitions to split the data into. Increase the number of partitions based on your Hadoop cluster resources to achieve a better throughput.

Tune the Spark Engine

You can tune the following Spark parameters to optimize the performance:

spark.executor.memory

The amount of memory that is used in each executor process.

The following table describes the tuning recommendations for the Spark parameters according to the size of the data set:

Data Set	Recommended Value
Default	6 GB
Small	3 GB
Medium	3 GB
Large	3 GB

spark.executor.cores

The number of cores that is used in each executor process.

The following table describes the tuning recommendations for the Spark parameters according to the size of the data set:

Data Set	Recommended Value
Default	2
Small	1
Medium	1
Large	1

spark.executor.instances

The count of executors to run a process if dynamic allocation is enabled.

The following table describes the tuning recommendations for the Spark parameters according to the size of the data set:

Data Set	Recommended Value
Default	10
Small	3
Medium	30
Large	150

spark.driver.memory

The amount of memory that is used for the driver process when the `SparkContext` parameter is initialized.

The following table describes the tuning recommendations for the Spark parameters according to the size of the data set:

Data Set	Recommended Value
Default	4 GB
Small	4 GB
Medium	4 GB
Large	4 GB

Note: Any changes to the Spark parameters have to be done in the cluster configuration object. You need not restart Data Integration Service post the changes. The changes are expected to be picked automatically for the subsequent runs.

In a small, medium, or large data set, you might get the following error:

```
Container is running beyond physical memory limits.
```

To resolve this error, increase the value of `spark.executor.memory` parameter from 2GB to 4GB.

Troubleshooting Spark Tuning Parameters

Use the following section to troubleshoot errors while tuning the Spark parameters:

ERROR: "Container is running beyond physical memory limits in Dataproc cluster"

To resolve this error, increase the `spark.executor.memory` parameter from 3 GB to 4 GB for a small, medium, or large data sets.

Authors

Akanksha Gauniyal

Anupam Nayak